

Unrolled List

–Destroy List

–Space Complexity Analysis and Algorithm

Program:

```
void destroyList() {
    if (head == NULL) {
        cout << "The list is already empty.\n";
        return;
    }

    struct Node* temp = head;
    while (temp != NULL) {
        struct Node* nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }

    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}

... •

    case 5:
        destroyList();
        break;
```

Space Complexity Analysis

1. Input Data Space

The input space measures the memory required to hold the data that the algorithm interacts with before it begins processing.

A. Parametric Input Space: $O(1)$

This refers exclusively to the memory needed for the arguments passed directly into the function signature.

- *The function signature is **void destroyList()**.*
- *It accepts absolutely zero parameters.*
- *Because no values are passed into the function, it requires zero extra memory for arguments.*
- **Result:** *The parametric input space is strictly $O(1)$ (Constant Space).*

B. Full Data Holistic Space: $O(N)$ or $O(n)$

This measures the memory consumed by the actual data structure the algorithm operates on.

- *The function interacts with the global unrolled linked list consisting of **N** node blocks to store **n** active elements.*
- *Even though the goal of the function is to delete this memory, at the exact moment the function is called, the data structure fully exists in the computer's RAM.*
- *Because the total memory occupied by the list scales linearly with the number of elements, the holistic space requirement at the start of the function is proportional to the size of the list.*
- **Result:** *The holistic input space is $O(N)$ (which is mathematically equivalent to $O(n)$).*

2. Auxiliary Space Complexity: $O(1)$

Auxiliary Space measures the temporary, extra memory that the algorithm uses exclusively to perform its task, entirely separate from the input data.

Let's trace every single local variable created inside the `destroyList` block:

- *One traversal pointer: `struct Node * temp = head;` declared at the beginning of the function.*
- *One temporary holding pointer: `struct Node * nextNode = temp → next;` declared inside the `while` loop to safely hold the next address before the current one is deleted.*

Key Observations:

1. **Memory Destruction, Not Allocation:** *The function explicitly uses `free(temp)` to deallocate memory. It never uses `malloc` or `new` to create new structures.*
 2. **No Recursion:** *The function uses a simple, iterative `while` loop.*
- The Call Stack does not grow; it remains at a depth of 1.*
-

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The for loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

*Inside the loop, the **malloc** function is called during every single iteration.*

***malloc** physically claims a brand – new*

block of memory from the computer's heap.

3. Total Memory Consumed

*If **sizeof(struct Node)** takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:*

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N, the complexity is $O(N)$.

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N *  
sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

For Academic Answer = Auxiliary Space = $O(1)$.

*"As we often cannot control the size of the input data, but we
can control how much extra memory of our algorithm
creates to process it."*

Total Space Complexity or Total memory footprint:
*= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space*

$$= (O(1) + O(N)) + O(1)$$

$$= O(N).$$

or,

$$(O(1) + O(n)) + O(1)$$

$$= O(n).$$

Key Notation Conventions

- **START**: Represents the global **head** pointer.
- **LINK[PTR]**: Represents the **next** pointer of a specific node block.
- **FREE(PTR)**: Represents the **free()** function used to deallocate the memory back to the system.

Algorithm: UnrolledDESTROYList()

1. [CHECK IF LIST IS EMPTY]

- IF START = NULL, Then:
 - Write: "The list is already empty."
 - RETURN
- [End of If structure.]

2. [INITIALIZE TRAVERSAL POINTER]

- SET TEMP_PTR := START

3. [OUTER LOOP: NODE TRAVERSAL AND DEALLOCATION]

- REPEAT WHILE TEMP_PTR ≠ NULL:

4. [SAVE NEXT ADDRESS]

- SET NEXT_PTR := LINK[TEMP_PTR] $\left(\begin{array}{l} \text{Temporarily saves the address} \\ \text{of the next node block so it is} \\ \text{not lost upon deletion} \end{array} \right)$

5. [FREE MEMORY]

- FREE(TEMP_PTR) $\left(\begin{array}{l} \text{Deallocates the current node,} \\ \text{including its array of elements and} \\ \text{metadata, all at once} \end{array} \right)$

6. [ADVANCE POINTER]

- SET TEMP_PTR := NEXT_PTR $\left(\begin{array}{l} \text{Moves the pointer forward} \\ \text{to the saved address} \end{array} \right)$
- [End of Node Traversal Loop]

7. [RESET GLOBAL POINTER]

- SET START := NULL $\left(\begin{array}{l} \text{Ensures the global head pointer is} \\ \text{safely nullified to prevent} \\ \text{dangling pointer errors} \end{array} \right)$

8. Write: "The entire list has been destroyed and memory freed."

9. *[END OF ALGORITHM]*
